

Objetivo del tema

En este tema aprenderás las bases del lenguaje de programación Kotlin.

5.1 ¿Kotlin?

Kotlin es un lenguaje de programación multiplataforma, tipado, de alto nivel y de propósito general con inferencia de tipos. Está diseñado para ser totalmente interoperable con **Java**, hasta el punto que un IDE puede traducir un código de un lenguaje a otro automáticamente. La versión JVM de la biblioteca estándar de Kotlin depende de la Java Class Library, pero la inferencia de tipos permite que su sintaxis sea más concisa. Kotlin se dirige principalmente a la JVM, pero también compila a JavaScript para aplicaciones web como React. Además, desde el lanzamiento de Android Studio 3.0 en octubre de 2017, Kotlin se ha incluido como alternativa al compilador Java estándar.

Esta es la definición que te puede dar una IA. Con lo que tienes que quedarte es que Kotlin, aunque no es necesariamente más rápido o eficiente que Java, sí es mucho **más eficiente** en términos de **desarrollo, mantenimiento y productividad**.

No, no es probable que sustituya a Java el año que viene, pero Google ya ha dicho que es su lenguaje preferido para las nuevas aplicaciones. ¿A medio y largo plazo? No se sabe. Java sigue estando por todas partes... pero Kotlin es el rey en el desarrollo móvil.

Importante – ¿Vamos a aprender Kotlin en clase?

No. Lo vas a aprender por tu cuenta. Si sabes Java, es fácil aprender Kotlin. La sintaxis de Kotlin a veces puede parecernos rara, pero en el fondo, es como si fuese Java resumido. Tocaré practicar bastante. En este módulo vamos a hacer uso de la IA (y más con la Optativa) y pronto te darás cuenta que esta forma de trabajar no se parece en nada a lo que has estado haciendo hasta ahora.

Sí. Te estamos diciendo que copiar, pegar y modificar código es lo que queremos que hagas.

No necesitas el Android Studio para programar Kotlin. Puedes usar el eclipse (si te bajas los paquetes correctos) para hacer los ejercicios que aquí te planteamos. Aconsejo mejor el **IntelliJ IDEA** porque la forma que tiene de organizar los proyectos y el código se asemeja a la de **Android Studio**. Basta con bajarse la versión de prueba, o si se desea, crearse una cuenta con licencia de estudiante (para tu uso personal).

5.2 Qué NO es igual en Kotlin

Kotlin no es 100% igual a Java. Pero puedes leer su código con facilidad. Mira los ejemplos si quieres, y verás que no es *tan diferente* de Java.

Nullable / No-Nullable

Kotlin diferencia las variables en dos tipos:

- Que SÍ pueden ser Nulas.
- Que NO pueden ser nulas.

Si no especificas nada, la variable es no-nullables. Es decir, que NO PUEDE tener el valor null. Si quieres que esa variable pueda tener null, tienes que usar “?”. Esto también se utiliza para hacer comprobaciones de nulo, etc.

¿Por qué nos restringen el uso del Null? Por multitud de motivos. Para empezar, evita que definas una variable y no la uses. Segundo, genera un error automáticamente cuando intentas asignar Null a una variable que es no-nullable. Y tercero, evita al programador estar continuamente preguntando si algo es Null o no es Null, ahorrando escribir código innecesario.

Var y Val

Se utiliza **var** y **val** a la hora de declarar variables. La diferencia entre ellas es que:

- **var** permite que el valor de la variable cambie.
- **val** no pueden cambiar nunca.

Pero cuidado, que **val** no es lo mismo que **static** de Java. Kotlin no usa variables estáticas.

Tipos de datos

Kotlin es capaz de **adivinar** por sí mismo el tipo de datos de las variables en una asignación. NO necesitas especificarlo absolutamente todo como en Java. Mira por ejemplo la imagen: no le estamos diciendo en ningún momento el tipo de dato de birth, pero el IDE no te da error. Simplemente ‘adivina’ que es un String.

```
val name: String = "John"
val birth = 1975
```

Clases, constructores, getters y setters

Kotlin permite hacer cosas como definir en una sola línea de texto la clase y su constructor. Permite incluso que se definan clases vacías, pero que en realidad tengan muchos métodos “invisibles”. Lo iremos viendo. Por el momento, recuerda solamente que no hace falta hacer getters y setters. Kotlin ya asume que existen.

5.3 Programando con Kotlin

Vamos a empezar con ejemplos sencillos. Sigue los siguientes pasos:

1. Accede a <https://www.w3schools.com/KOTLIN/index.php>
2. Verás que es una página con un tutorial básico de Kotlin,
3. Revisa desde Kotlin Home hasta Kotlin Functions. Te resumen los elementos básicos del lenguaje de programación: variables, bucles, funciones...
4. Accede a los ejemplos:
 - a. Lee y ejecuta las clases **Main01.kt** hasta **Main07.kt**
 - b. Observa los comentarios y haz pruebas por tu cuenta.
5. Realiza los ejercicios propuestos a continuación.

Ejercicio 2

Crea un programa Kotlin que solicita un usuario y un password. Si ambos son 'admin', le mostramos un mensaje de bienvenida. Si no, le mostramos un mensaje de error.

Ejercicio 3

Modifica el ejercicio anterior para que permita tres reintentos.

Ejercicio 4

Modifica el ejercicio anterior para que el programa, una vez haya entrado el usuario, muestre las siguientes opciones. Guardaremos los nombres en un array de tamaño 10.

- 1) Nuevo nombre
- 2) Mostrar nombres
- 3) Buscar nombre
- 4) Modificar nombre
- 5) Borrar nombre

Ejercicio 5

Modifica el ejercicio anterior para que el array tenga que estar siempre ordenado de forma ascendente por nombre. Por lo tanto, al insertar, borrar o modificar un nombre, será preciso reordenar el array.

Ejercicio 6 – Cartelera de Cine

Se desea desarrollar una aplicación para gestionar la cartelera de películas del cine de Elorrieta. En esta primera versión, sólo vamos a tener en cuenta los fines de semana. Durante los sábados, se proyectan películas durante 8 horas; los domingos durante 6 horas. Nuestro programa deberá permitir rellenar de películas de la cartelera, teniendo en cuenta que no todas las películas duran lo mismo, y que no podemos exceder las horas máximas de proyección.

Las películas tienen todas un título y una duración. Deberán estar organizadas en géneros, de forma que una película (*Memorias de Ana Frank*) deberá aparecernos en el programa cuando seleccionemos su género correspondiente (*Drama*).

Paso 1 - Al arrancar el programa, se mostrará un mensaje de bienvenida. Al pasar 3 segundos, se solicitará al usuario del programa que se identifique con un nombre y su contraseña.

- Si el usuario y la contraseña son correctos, el programa deberá mostrar la lista de géneros de películas disponibles (**Paso 2**).
- Si el usuario, la contraseña o ambos **no** son correctos, se mostrará un mensaje de error. Se deberá volver a solicitar el usuario y la contraseña de nuevo.
 - Restricción: al introducir tres veces mal el usuario y/o la contraseña, el programa mostrará el mensaje “Usuario bloqueado” y tras tres segundos, se volverá a mostrar el mensaje de bienvenida (**Paso 1**).

Paso 2 - Se mostrará un menú de géneros. Además, se mostrarán también: las películas que ya han sido seleccionadas por el usuario (de haberlas) y el tiempo restante que nos queda sin asignar del sábado y del domingo. El menú de géneros deberá tener cinco opciones y será similar a esto.

- 0. Salir
- 1.Drama
- 2.Comedia
- 3.Terror
- 4.Ciencia Ficción
- 5.Ver resumen

Si se selecciona la opción Salir el programa terminará inmediatamente.

- Restricción: el programa mostrará el mensaje “Adiós” y tras tres segundos, se volverá a mostrar el mensaje de bienvenida (**Paso 1**).

Como puedes ver más arriba, hay cuatro géneros: Drama, Comedia, Terror y Ciencia Ficción. Al seleccionar un género, se nos mostrarán las películas de ese género.

Por tanto, al seleccionar una opción:

- Si se escoge una opción del menú que no existe, se mostrará un mensaje de error. Se volverá a mostrar el menú y el usuario deberá escoger otra opción.
- Si se escoge un género, se pasará a mostrar el listado de películas de ese género **(Paso 3)**.
- Si se escoge la opción Ver resumen, se pasará a mostrar el resumen de la cartelera **(Paso 4)**.

Paso 3 - Al seleccionar un género, se mostrará el menú de películas con su título y duración. El menú siempre tendrá una opción de Atrás; pero el número de películas mostradas variará. No se van a mostrar todas las películas de un género, solamente aquellas cuya duración sea igual o inferior al tiempo que tengamos disponible para el sábado o el domingo, según el día en el que estemos añadiendo películas. Si se selecciona la opción Atrás volverá a mostrarse la lista de géneros **(Paso 2)**.

- Si el usuario escoge una película de la lista, se descontará del tiempo disponible de ese día la duración de la película. Se regresará a la lista de géneros **(Paso 2)**.
 - Restricción: en un mismo día, no se puede proyectar más de una película del mismo género. Si esto ocurriese, se deberá de mostrar un mensaje de error. Se volverá a mostrar el menú y el usuario deberá escoger otra opción.
 - Restricción: Por defecto, cuando seleccionas una película, ésta se asigna al sábado hasta que no quede tiempo para más películas. Sólo entonces, se pasará a asignar películas al domingo automáticamente. Si no hubiese espacio para más películas el domingo, se deberá de mostrar un mensaje de error. Se volverá a mostrar el menú y el usuario deberá escoger otra opción.
- Si el usuario escoge opción del menú que no existe, se mostrará un mensaje de error. Se volverá a mostrar el menú y el usuario deberá escoger otra opción.

Paso 4 - En el resumen de la cartelera se muestran todas las películas programadas para el sábado y el domingo. Se hace también una comprobación extra: si hay tiempo disponible en el sábado o en el domingo, y en él podemos asignar una película (cualquiera), debemos de asignarla.

- Si no hay tiempo disponible ni en el sábado ni en el domingo, se mostrará un mensaje de aviso y el programa continuará.
- Si hay tiempo disponible para el sábado y/o el domingo, se mostrará un mensaje de aviso. Ahora el usuario tiene que completar los huecos.

A la hora de completar los huecos de nuestra cartelera, se mostrarán todas las películas cuya duración sea igual o inferior al tiempo disponible restante de ese día, y debemos permitir seleccionar y asignar una de ellas tal y como lo hacemos en el Paso 3. El usuario estará obligado a ir seleccionando películas hasta que no le quede tiempo disponible en ningún día. Nótese que nos da igual que el género de la película se repita o que ya esté seleccionada. Basta con que la duración sea menor o inferior al tiempo disponible.

Una vez que hemos confirmado que no hay huecos, se muestra un resumen de la cartelera: Para cada día, las películas seleccionadas con sus títulos y su duración, el tiempo disponible restante y el tiempo de proyección total real.

Para terminar, se le preguntará al usuario si está de acuerdo.

- En caso afirmativo, se muestra un mensaje informativo y tras tres segundos se volverá a mostrar el mensaje de bienvenida **(Paso 1)**.
- En caso negativo, se muestra un mensaje informativo, se borrarán todas las películas asignadas al sábado y el domingo. Se regresará de nuevo a la lista de géneros **(Paso 2)**.

🔔 Importante – ¿Pero qué es esto?

Si, es el Reto 1 de la asignatura de Programación de 1º. Lo tienes resuelto en el ejemplo **Main08.kt**. No hace falta que lo hagas entero. Pero para practicar un poco está bien. Mira la solución y verás que, pese a lo largo del código, lo entiendes perfectamente. De eso va el ejercicio.

5.4 La Programación Orientada a Objetos

POO significa **Programación Orientada a Objetos**. No vamos a extendernos demasiado en esto porque ya habéis trabajado con Java, que también es POO. Digamos que la programación orientada a objetos consiste en crear objetos que contienen tanto los datos como los métodos de un programa.

Todo en **Kotlin** son clases y objetos. Es decir, que NO existen los tipos de datos primitivos en Kotlin. Crear una clase en Kotlin es sencillo, puesto que sólo se necesita utilizar la palabra reservada **class**.

```
class Car {  
    var brand = ""  
    var model = ""  
    var year = 0  
}
```

Como ves, faltan la mayoría de las palabras reservadas usadas en Java. En Kotlin no son necesarias. Eso sí:

- Todas las propiedades y funciones **son públicas** a menos que especifiques lo contrario.
- Todo lo que hemos explicado antes de las variables se sigue aplicando,
- Por lo demás, es como Java.

Kotlin utiliza **constructores** para inicializar objetos. Su sintaxis es diferente a las de Java y confunde bastante.

Un constructor se define mediante el uso de dos paréntesis () después del nombre de la clase, donde especificas las propiedades. El constructor inicializará las propiedades cuando crees un objeto de una clase.

Es decir, en una sola línea tienes la clase, el constructor y su inicialización:

```
class Car(var brand: String, var model: String, var year: Int)
```

Kotlin también dispone de mecanismos para implementar la **herencia**. Funciona de forma similar a Java, dado que se heredan las propiedades y las funciones de la clase padre. Para indicar una herencia se utiliza ":"

```
class MyChildClass: MyParentClass() {
```

El tema de la herencia es más complicado en realidad. Por defecto, **Kotlin** asume que todas las clases, funciones y atributos son **finales**, es decir, que no puedes extender de ellos o sobrescribirlos (**override**). Para permitirlo, debemos marcar las clases, métodos o atributos como abiertas (**open**).

Por tanto, en el caso anterior, para que PODAMOS heredar de ella la clase MyParentClass quedaría así:

```
open class MyParentClass {  
    val x = 5  
}
```

Vamos a seguir con ejemplos sencillos. Sigue los siguientes pasos:

1. Accede a <https://www.w3schools.com/KOTLIN/index.php>
2. Verás que es una página con un tutorial básico de Kotlin,
3. Revisa desde Kotlin Classes. Te resumen los elementos básicos del lenguaje de programación: variables, bucles, funciones...
4. Accede a los ejemplos:
 - a. Lee y ejecuta las clases **Main09.kt** hasta **Main12.kt**
 - b. Observa los comentarios y haz pruebas por tu cuenta.
5. Realiza los ejercicios propuestos a continuación.

Ejercicio 7

Modifica el ejercicio 5 para que en lugar de nombres tengamos Alumnos con Nombre, Apellido, Edad y Curso.

Ejercicio 8

Modifica el ejercicio anterior para que cada Alumno tenga Nota (Nota, Nombre asignatura). Añade al menú las siguientes opciones:

- 1) Añadir nueva nota
- 2) Ver todas las notas de alumno
- 3) Buscar nota de asignatura
- 4) Modificar nota de asignatura
- 5) Borrar nota de asignatura

Ejercicio 9

Crea un programa en Kotlin que lea un fichero de texto y lo muestre por pantalla. Busca en internet la información que necesites.

Lo tienes ya resuelto en **FileReader.kt**.